

RÉSUMÉ DE COURS

Systèmes de Contrôle de Version & Git

1. Introduction aux Systèmes de Contrôle de Version

Un **système de contrôle de version (SCV / VCS)** est un outil qui enregistre l'historique des modifications d'un ensemble de fichiers. Il permet de revenir à n'importe quelle version, de comprendre qui a fait quoi et quand, et de collaborer sans conflit.

Pourquoi utiliser un VCS ?

- Historique complet — revenez à n'importe quelle version passée
- Collaboration — plusieurs développeurs travaillent simultanément
- Sauvegarde — le dépôt constitue une sauvegarde de tout le travail
- Traçabilité — retrouvez quel commit a introduit un bug
-

Les trois grandes familles

Famille	Caractéristiques
Local (RCS, SCCS)	Un seul poste, sans réseau — très limité
Centralisé (SVN, CVS, Perforce)	Serveur unique, tous les clients s'y connectent
Distribué (Git, Mercurial)	Chaque clone est un dépôt complet et autonome

2. Système de Contrôle de Version Centralisé

Dans un SCV centralisé, il existe un unique serveur central contenant tout l'historique. Les développeurs récupèrent (checkout) une copie et y soumettent (commit) directement leurs modifications.

Architecture

Dev A <--> Serveur Central <--> Dev B

Opérations principales

Commande	Description
<code>checkout</code>	Récupère la dernière version depuis le serveur
<code>update</code>	Met à jour la copie locale avec les changements distants
<code>commit</code>	Envoie directement les modifications au serveur central
<code>lock</code>	Verrouille un fichier pour éviter les conflits

Avantages	Inconvénients
→ Simple à comprendre et administrer → Un seul endroit à sauvegarder → Contrôle d'accès centralisé facile → Historique global cohérent	→ Point unique de défaillance : si le serveur tombe, tout s'arrête → Travail hors ligne très limité → Les commits nécessitent un accès réseau → Branching lourd et coûteux

Scénario de panne : Si le serveur SVN tombe, les développeurs ne peuvent plus commiter, consulter l'historique, ni partager. Toute collaboration est bloquée jusqu'au retour du serveur.

3. Git — Système Distribué

Git a été créé par **Linus Torvalds** en **2005** pour le noyau Linux. Chaque développeur possède un clone complet du dépôt avec tout l'historique. Il n'y a pas de point unique de défaillance.

Scénario de panne avec Git : Si GitHub tombe, les développeurs continuent de commiter localement, consultent l'historique complet, créent des branches, et synchronisent dès le retour du serveur. Aucune interruption.

Concepts fondamentaux

Concept	Explication
SHA-1 / Hash	Chaque commit est identifié par un hash unique de 40 caractères garantissant l'intégrité

Snapshots	Git stocke des snapshots complets, pas des deltas. Fichiers inchangés référencés.
HEAD	Pointeur spécial indiquant la position courante dans l'historique
Objets Git	4 types : blob (fichier), tree (dossier), commit, tag

Configuration initiale

obligatoire avant le premier commit :

```
git config --global user.name "Votre Nom"
git config --global user.email "email@example.com"
git config --list
```

4. Les 3 Zones de Git — Concept

Comprendre les 3 zones est **essentiel**. Tous les problèmes Git viennent d'une confusion entre elles.

Working Directory Vos fichiers sur disque. Zone de travail brute. <i>Modifié / Untracked</i>	Index (Staging Area) Zone tampon. Choisissez ce qui fera partie du prochain commit. <i>Staged</i>	Repository (.git) Base de données permanente. Tous les commits et l'historique. <i>Committed</i>
---	--	---

Working Dir --git add--> **Index** --git commit--> **Repository**

Etats d'un fichier dans Git

Etat	Signification
Untracked	Fichier nouveau, inconnu de Git -> git add pour commencer
Modified	Suivi par Git mais modifié, pas encore stage -> git add
Staged	Dans l'index, prêt pour le prochain commit -> git commit
Committed	Sauvegarde dans le dépôt Git
Ignored	Explicitement ignore via .gitignore

5. Commandes de Base

Initialisation & Clonage

git init # Créer un depot dans le dossier courant

git init mon-projet # Créée le dossier et initialise

git clone https://url # Cloner un dépôt distant

Observer l'état

git status # Etat du working dir et de l'index

git status -s # Format court

git log # Historique des commits

git log --oneline # Une ligne par commit

git log --oneline --graph --all # Graphe de toutes les branches

git show abc123 # Détails d'un commit spécifique

Ajouter & Commiter

git add fichier.txt # Ajouter un fichier à l'index

git add . # Ajouter tous les fichiers modifiés

git commit -m "message" # Commit avec message en ligne

git commit -am "message" # Add + commit (fichiers déjà trackers)

git commit --amend # Modifié le dernier commit

Comparer les différences (git diff)

Commande	Description
git diff	WD vs Index — modifications NON stages
git diff --staged	Index vs dernier commit
git diff HEAD	WD vs dernier commit
git diff branche1 branche2	Différences entre deux branches

Règle : git diff --staged = 'ce que le prochain commit va contenir'. La commande à utiliser avant de commiter.

Gestion des fichiers sensibles (.gitignore)

-Contenu du fichier .gitignore :

```
dev.env          # Ignore ce fichier spécifique
*.env            # Ignore tous les fichiers .env
node_modules/    # Ignore ce dossier entier
```

-Si dev.env a déjà été commiter par erreur :

```
git rm --cached dev.env      # Retire du suivi, conserve localement
echo "dev.env" >> .gitignore
git commit -m "Remove dev.env from tracking"
```

6. Branches & Fusion

Une branche dans Git est un **pointeur léger vers un commit**. La créer est quasi instantané. C'est l'une des forces majeures de Git.

Gestion des branches

```
git branch          # Lister les branches locales
git branch -a       # Lister toutes (locales + distantes)
git branch feature   # Créer une branche (sans basculer)
git checkout -b feature # * Créer ET basculer (1 seule commande)
git switch -c feature # Équivalent moderne (Git 2.23+)
git branch -d feature # Supprimer
git branch -D feature # Forcer la suppression
```

Fusion - Merge vs Rebase

```
git checkout main
git merge feature      # Merge classique
git merge --no-ff feature # Force un commit de fusion
–Mettre à jour sa branche SANS commit de fusion :
git checkout feature
git rebase main
```

git merge	git rebase
------------------	-------------------

- Préserver l'historique complet
- Créer un commit de fusion
- Garde une trace des branches
- Préfère en équipe — non-destructif
- Ne réécrit pas l'historique

- Rejoue les commits sur une nouvelle base
- Historique linéaire et propre
- Écrit l'historique avertissement
- Ne jamais rebaser des commits déjà poussés
- Utile en local avant de partager

Règle d'or du rebase : Ne jamais rebaser des commits qui ont déjà été pushés sur un dépôt partagé. Cela réécrit l'historique et crée des conflits pour les autres développeurs.

7. Conflits & Resolution

Un conflit survient quand deux branches ont modifié la **même zone du même fichier**. Git ne peut pas décider automatiquement laquelle prendre.

Les marqueurs de conflit

```
<<<<<<< HEAD
hello
=====
bye
>>>>>>> feature
```

Marqueur	Signification
<<<<<<< HEAD	Début du conflit. Ce qui suit jusqu'à ===== est le contenu de VOTRE branche courante (main)
=====	Séparateur entre les deux versions
>>>>>>> feature	Fin du conflit. Ce qui est entre ===== et ici est le contenu de la branche entrante

Procédure complète de résolution

1. Git vous informe : CONFLICT (content): Merge conflict in fichier
2. Voir les fichiers en conflit : git status (en rouge)
3. Éditer manuellement — supprimer les marqueurs <<<, ==, >>> et garder le code voulu
4. Marquer comme résolu : git add fichier
5. Finaliser le merge : git commit

En cas de regret, annuler le merge en cours :

git merge --abort

8. Depot Distant (Remote)

Gestion des remotes

git remote -v # Lister les dépôts distants

git remote add origin URL # Ajouter un remote nomme 'origin'

git remote remove origin # Supprimer un remote

Push / Fetch / Pull

git push origin main # Envoyer la branche main vers origin

git push -u origin main # Push + definir le upstream (tracking)

git fetch # Récupère SANS fusionner

git fetch origin # Récupère depuis origin

git pull # fetch + merge automatique

git pull --rebase # fetch + rebase (historique linéaire)

git fetch	git pull
→ Récupérer les données distantes → Ne fusionne PAS automatiquement → Vous pouvez inspecter avant de fusionner → Sur a utiliser à tout moment	→ = fetch + merge en une commande → Fusionne immédiatement → Peut créer des conflits sans préavis → Moins de contrôle

Supprimer des branches

git branch -d feature # Supprimer la branche LOCALE

git push origin --delete feature # Supprimer la branche DISTANTE

9. Commandes Avancees

Reset — Annuler des commits

Commande	Index	WD	Usage
----------	-------	----	-------

<code>git reset --soft HEAD~1</code>	Conservé	Conservé	Défaire le commit, garder tout stage
<code>git reset --mixed HEAD~1</code>	Vidé	Conservé	Défaire le commit et l'index (défaut)
<code>git reset --hard HEAD~1</code>	Vidé	PERDU	Efface tout

Revert — Annuler sans réécrire l'historique

`git revert HEAD` # Annule le dernier commit en créant **UN NOUVEAU** commit

`git revert HEAD~2` # Annule le commit 2 positions avant HEAD

`git revert abc123` # Annuler un commit spécifique par son hash

`git revert --no-commit HEAD` # Prépare l'annulation sans commiter

git reset --hard	git revert
→ Supprime les commits de l'historique → Modifié l'historique — dangereux si partage → Nécessite un force push → Les autres devs auront des conflits	→ Créer un nouveau commit d'annulation → Historique préserve intégralement → Push normal, pas de force push → Les autres reçoivent juste un nouveau commit

Stash — Sauvegarder le travail en cours

`git stash` # Sauvegarde WD + index

`git stash -u` # Inclut les fichiers untracked

`git stash -a` # Inclut TOUS (même ignorés)

`git stash save "description"` # Sauvegarde avec message

`git stash list` # Lister toutes les sauvegardes

`git stash pop` # Applique le dernier stash et le supprime

`git stash apply` # Applique sans supprimer

`git stash apply stash@{2}` # Applique un stash spécifique

`git stash drop` # Supprime le dernier stash

Historique & Exploration

Commande	Description
<code>git log --oneline --graph --all</code>	Graphe condensé de toutes les branches *
<code>git log --author='Nom'</code>	Filtrer par auteur
<code>git log --since='2024-01-01'</code>	Filtrer par date
<code>git blame fichier.txt</code>	Qui a modifié chaque ligne
<code>git reflog</code>	Historique complet de HEAD (récupérer après reset)
<code>git bisect start</code>	Trouver le commit qui a introduit un bug

10. Cheatsheet Complet

Configuration & Initialisation

<code>git config --global user.name "Nom"</code>	Définir le nom d'utilisateur global
<code>git config --global user.email "e@x"</code>	Définir l'email global
<code>git config --list</code>	Voir toute la configuration
<code>git init</code>	Créer un nouveau dépôt local
<code>git clone <url></code>	Cloner un dépôt distant

Staging & Commits

<code>git status</code>	Etat du working directory et de l'index
<code>git add <fichier></code>	Ajouter un fichier à l'index
<code>git add .</code>	Ajouter tous les fichiers modifiés
<code>git commit -m "message"</code>	Créer un commit avec un message
<code>git commit --amend</code>	Modifier le dernier commit
<code>git rm --cached <fichier></code>	Retirer de l'index sans supprimer localement
<code>git diff --staged</code>	Voir ce qui sera commiter

Branches

<code>git branch</code>	Lister les branches locales
-------------------------	-----------------------------

<code>git checkout -b <branche></code>	Créer et basculer sur une nouvelle branche *
<code>git merge <branche></code>	Fusionner une branche dans la courante
<code>git rebase <branche></code>	Rebaser (linéarisé l'historique)
<code>git branch -d <branche></code>	Supprimer une branche locale
<code>git push origin --delete <branche></code>	Supprimer une branche distante *

Annulation

<code>git reset --soft HEAD~1</code>	Defaire le commit, garder les fichiers stages
<code>git reset --mixed HEAD~1</code>	Defaire le commit et le stage (par défaut)
<code>git reset --hard HEAD~1</code>	Efface tout, modifications perdues avertissement
<code>git revert HEAD</code>	Annule en creant un nouveau commit (sur) *
<code>git restore <fichier></code>	Restaurer un fichier depuis l'index
<code>git restore --staged <fichier></code>	Retirer de l'index (unstage)

Stash & Remote

<code>git stash</code>	Sauvegarder le travail en cours
<code>git stash -a</code>	Inclure TOUS les fichiers
<code>git stash list</code>	Lister toutes les sauvegardes
<code>git stash pop</code>	Appliquer et supprimer le dernier stash
<code>git fetch</code>	Récupérer les données distantes sans fusionner *
<code>git pull</code>	fetch + merge automatique
<code>git push origin <branche></code>	Envoyer une branche vers le distant
<code>git log --oneline --graph --all</code>	Historique visuel de toutes les branches
<code>git rebase -i HEAD~N</code>	Rebase interactif sur les N derniers commits
<code>git cherry-pick <hash></code>	Appliquer un commit sur la branche courante
<code>git reflog</code>	Historique complet de HEAD